

C2

What does it actually cost?

~35 min + ~28 min after the break · two forces · the inversion · break-even · the calculator

BY THE END OF THIS CAPSULE YOU WILL BE ABLE TO

Put a defensible number on a use case — cost per successful task, the break-even success rate, and how much the answer moves when you are wrong — and build the calculator that produces it.

Token prices fell about 1,000× in three years. What happened to enterprise AI bills?

A · Fell by roughly the same factor

B · Fell, but much less than 1,000×

C · Stayed about flat

D · Went up

Think about what a team does when a thing it wanted gets cheap.

Two forces, pulling opposite ways

Most people know about the first one. The second is why the invoice went up anyway.

Force 1 — the price of a fixed capability collapses. a16z: about 10× a year. GPT-3-quality output went from \$60 per million tokens in Nov 2021 to about \$0.06 three years later — 1,000×. Stanford's AI Index put GPT-3.5-level at \$20.00 → \$0.07, over 280×.

The catch: both are measured at FIXED capability. Nobody buys the 2021 capability.

Force 2 — the tokens a task consumes explode. Reasoning models emit about 8× more tokens on the same questions, with output length growing ~5× a year. Anthropic: agents ≈ 4× a chat, multi-agent ≈ 15×.

Energy says it too: a short GPT-4o query ≈ 0.42 Wh; GPT-5 at high reasoning ≈ 17.15 Wh — about 40× for the same question.

$\text{bill} = \text{price per token} \downarrow \times \text{tokens per task} \uparrow \times \text{number of tasks} \uparrow$ — which one do you actually control?

Takeaway Price per token fell. Tokens per task rose. The third term is the one you choose.

The unit you quote decides the answer

One of these is an input price. The other is a business cost. Only one of them decides anything.

Dollars per million tokens — an INPUT PRICE. Useful for comparing vendors. Useless for comparing architectures, because it says nothing about how many tokens a task takes or how often the task has to be redone.

It is: the price of the raw material, not the cost of the finished thing.

Cost per SUCCESSFUL task — a BUSINESS COST. Token and tool cost, plus the expected cost of handling the failures. This is the number a sponsor can act on, and the one Artificial Analysis moved to reporting in June 2026.

Watch what happens next: the two units are about to rank these four in OPPOSITE orders.

v1 • one call

v2 • + retrieval

v3 • 6-turn agent, cheap model

v4 • same agent, frontier model

Takeaway Quote cost per successful task. Everything else is a raw-material price.

One use case, four architectures

A support-triage assistant, 50,000 tickets a month. Cheap tier \$0.10 / \$0.40 • frontier tier \$5 / \$25 per million tokens.

Architecture	Tokens per task	Token cost per task	Per month • 50,000 tasks	vs v1
v1 • one call	1,500 in • 400 out	$\$0.00015 + \$0.00016 = \$0.00031$	\$15.50	base

Takeaway On token price the ranking is obvious: $v1 < v2 < v3 < v4$.

One use case, four architectures

A support-triage assistant, 50,000 tickets a month. Cheap tier \$0.10 / \$0.40 • frontier tier \$5 / \$25 per million tokens.

Architecture	Tokens per task	Token cost per task	Per month • 50,000 tasks	vs v1
v1 • one call	1,500 in • 400 out	$\$0.00015 + \$0.00016 = \$0.00031$	\$15.50	base
v2 • + retrieval	5,500 in • 400 out	$\$0.00055 + \$0.00016 = \$0.00071$	\$35.50 (+ ~\$50 fixed)	2.3× +129%

Input 1,500 → 5,500: the top-k retrieved chunks are pasted into the prompt.
And the + ~\$50 is not tokens at all — which layer of the Class 2 stack sends that bill?

Takeaway On token price the ranking is obvious: $v1 < v2 < v3 < v4$.

One use case, four architectures

A support-triage assistant, 50,000 tickets a month. Cheap tier \$0.10 / \$0.40 • frontier tier \$5 / \$25 per million tokens.

Architecture	Tokens per task	Token cost per task	Per month • 50,000 tasks	vs v1
v1 • one call	1,500 in • 400 out	$\$0.00015 + \$0.00016 = \$0.00031$	\$15.50	base
v2 • + retrieval	5,500 in • 400 out	$\$0.00055 + \$0.00016 = \$0.00071$	\$35.50 (+ ~\$50 fixed)	2.3× +129%
v3 • 6-turn agent, cheap model	45,000 in • 2,400 out	$\$0.0045 + \$0.00096 = \$0.00546$	\$273	17.6× +1,660%

$$\text{input} = B \times T + D \times T(T-1)/2 \approx BT + \frac{1}{2}DT^2 \quad 5,500 \times 6 + 800 \times 15 = 33,000 + 12,000 = 45,000$$

Takeaway On token price the ranking is obvious: $v1 < v2 < v3 < v4$.

One use case, four architectures

A support-triage assistant, 50,000 tickets a month. Cheap tier \$0.10 / \$0.40 • frontier tier \$5 / \$25 per million tokens.

Architecture	Tokens per task	Token cost per task	Per month • 50,000 tasks	vs v1
v1 • one call	1,500 in • 400 out	$\$0.00015 + \$0.00016 = \$0.00031$	\$15.50	base
v2 • + retrieval	5,500 in • 400 out	$\$0.00055 + \$0.00016 = \$0.00071$	\$35.50 (+ ~\$50 fixed)	2.3× +129%
v3 • 6-turn agent, cheap model	45,000 in • 2,400 out	$\$0.0045 + \$0.00096 = \$0.00546$	\$273	17.6× +1,660%
v4 • same agent, frontier model	45,000 in • 2,400 out	$\$0.225 + \$0.06 = \$0.285$	\$14,250	920× +91,800%

Takeaway v4 is 52× v3 and 920× v1 — and we have not asked whether any of them works.

Now add the only column that matters

Every unresolved ticket falls to a human: eight minutes at a loaded US\$45 an hour — \$6.00 an escalation.

	Token cost	Resolves unaided	Expected human cost	Cost per RESOLVED ticket
v3 • cheap agent	\$0.00546 base	55%	$0.45 \times \$6.00 = \2.70	?
v4 • frontier agent	\$0.285 52×	80%	$0.20 \times \$6.00 = \1.20	?

The cost of failure is paid in human labour — and it outweighs anything you can do to the agent itself. Optimise it, permission it, gate it (Week 4: stop • ask • approve): a well-governed agent that still fails 45% of the time loses to one that fails less.

Takeaway Before I fill the last column — which one is cheaper to run?

Now add the only column that matters

The architecture that costs 52× more per token is 45% cheaper per resolved ticket.

	Token cost	Resolves unaided	Expected human cost	Cost per RESOLVED ticket
v3 • cheap agent	\$0.00546 base	55%	$0.45 \times \$6.00 = \2.70	\$2.71
v4 • frontier agent	\$0.285 52×	80%	$0.20 \times \$6.00 = \1.20	\$1.49 — 45% CHEAPER

On a 50,000-ticket queue, choosing on token price costs about \$61,000 a month — roughly \$730,000 a year.

Takeaway Cost per token and cost per successful task rank these in opposite orders.

Turn the argument into an engineering target

Once both options are priced, the disagreement stops being a matter of taste and becomes one equation.

Set them equal and solve. $\$0.00546 + (1 - p) \times \$6.00 = \$1.485 \rightarrow (1 - p) = 0.2466 \rightarrow p \approx 75.3\%$. The cheap agent wins only if its resolution rate reaches about 75%.

Which means: the question is no longer “which model” — it is “can we get v3 from 55% to 75%?”

And that is a question with an owner and a deadline. Better prompts, better tools, better retrieval, a tighter eval set. All of Weeks 3 and 4 are, in this light, a cost-reduction programme wearing a quality-improvement label.

The habit: always report the break-even alongside the answer. It is what ends the meeting.

Takeaway A break-even converts an opinion into a target somebody can be accountable for.

Your agent resolves 80% of tickets at \$1.485 each. You may do exactly ONE thing. Which saves the most per resolved ticket?

A · Halve your token price

B · Raise resolution from 80% to 85%

C · Cut the prompt from 5,500 to 2,750 tokens

D · Move from monthly to annual billing

Same v4 agent we just priced — an escalation costs \$6.00 of human time.

What actually moves the number

In order of leverage on this shape of problem — which is not the order teams usually work in.

Knob	What it does	Worth
1 • Raise the success rate	Shrinks the term that dominates cost per successful task	Largest, almost always
2 • Route by difficulty	Cheap model for the easy majority, escalate only hard cases	The tiers are 40–50× apart on input
3 • Cache aggressively	Cached input $\approx 0.1\times$ input; writes cost $1.25\times$ (5 min) or $2\times$ (1 hr)	Decisive in an agent loop
4 • Batch anything not interactive	A flat 50% discount across all three major vendors	Half the bill, for waiting
5 • Control the observation size	A tool returning 200 tokens instead of 2,000 changes the quadratic term	Compounds every turn
6 • Move work out of the model	Validation, arithmetic, lookups, formatting belong in code	Free, and cannot hallucinate

Why teams do the opposite: the token bill arrives monthly with a chart addressed to someone. The escalation cost is buried in an operations budget that predates the project.

Takeaway Work on the failure rate. Treat model-cost tuning as housekeeping, not strategy.

When the economics only work at scale

INSTEAD OF THE GUEST LECTURE

PRE-READ 3

The same use case can be obviously right for one firm and absurd for another. The difference is not ambition.

Fixed costs amortise. Variable costs never do. The pipeline, the eval harness, the infrastructure are built once and spread over every decision. Inference is per-call and grows linearly with usage, forever.

So the threshold is: volume high enough to amortise the fixed half, at a margin the variable half still leaves you.

Which is why size shows up in the survey data. 48% of firms above \$5bn revenue have begun scaling AI enterprise-wide, against 29% of firms under \$100m — the same fixed cost, spread over very different revenue.

And what the market is NOT solving: NVIDIA and six asset managers are assembling over US\$500bn to finance the COMPUTE layer — someone else's fixed cost, resold to you as price per token. Nobody is financing YOUR fixed half.

Bessemer's test, worth memorising: if the maths does not work at 10 customers, it will not work at 1,000. Variable costs do not amortise.

Takeaway Ask not “does this work?” but “does this work at OUR volume?”

More tokens is not more accuracy

The tempting inference from everything so far is that spending more buys more. The best evidence says otherwise.

Evidence	What it found	What to do
Holistic Agent Leaderboard (arXiv 2510.11977, Oct 2025) 21,730 rollouts • 9 models × 9 benchmarks • ~\$40,000	Higher reasoning effort reduced accuracy in the majority of runs	Treat reasoning effort as a tunable with an optimum, not a dial that only goes up
Princeton HAL, SWE-bench Verified	Claude 3.5 Sonnet 38.0% at \$67.09 • o1-mini 27.2% at \$366.81 — 5.5× the cost, 10.8 points worse	Evaluate on a cost-accuracy frontier, never accuracy alone
Artificial Analysis, Jun 2026	Moved to reporting cost, time and tokens PER TASK; comparable systems sit 20–45× apart on cost	Report the unit that matches the workload

Put reasoning effort in your eval sweep alongside temperature and model choice — the middle setting often wins on both axes.

Takeaway “We used the best model” is not an engineering decision. It is the absence of one.

What cost-to-serve really contains

INSTEAD OF THE GUEST LECTURE

PRE-READ 3

Everything so far has been tokens. Tokens are the part that is easy to see, and rarely the part that decides.

Cost line	Why it is missed	Rough shape
Retries and failed runs	You are billed for attempts that produced nothing usable. In a pilot someone just clicks run again — nobody logs it	At a 20% failure rate you pay ~1.25× the tokens
Human review and escalation	It lands in an operations budget that predates the project	Usually the largest single line
Integration and change	One-off, so it gets excluded from “run cost” — then recurs at every upgrade	Lumpy, and under-forecast
Support, monitoring, on-call	A pilot has none of it	Per-deployment, not per-call
Customer success and sales time	Not an engineering cost, so engineering never counts it	Real if you are selling it
The engineer who keeps it alive	Nobody budgets maintenance for something that “just runs”	Fractional, permanent

Bessemer, Feb 2026 — their cost-to-serve list is model inference, human-in-the-loop support, customer-success overhead and sales time.

Takeaway Tokens are the visible line. The invisible ones are usually bigger.

From pilot to product

INSTEAD OF THE GUEST LECTURE

PRE-READ 2

Not a product-management lecture — four things that change in the arithmetic the day a capability acquires customers.

Pricing shape has to change, and margin becomes a constraint. With variable cost of goods sold, per-seat pricing makes your heaviest user your least profitable one — hence outcome pricing, like Intercom's Fin at \$0.99 per resolved ticket. AI-native gross margins run 50–60% against 80–90% for classic software.

Consequence: at 52% margin you cannot fund the research intensity an 80% business can. It bounds the roadmap.

Ownership and evidence change too. A pilot has a sponsor; a product has someone who must FORECAST a cost they do not control, on vendor prices that move. And the eval set stops being yours — it becomes the thing you promise against, and regress on.

The tell: if nobody owns the forecast, it is still a pilot, whatever the roadmap says.

Bessemer's advice is the opposite of the obvious one: founders default to cost-plus and systematically undercharge — price on VALUE, because a cost-linked price hands the customer the upside of every efficiency you find (Pre-read 3).

Takeaway A pilot spends a budget. A product carries a margin, a price and a promise.

What to measure, and when — before you build

Design the measurement first. This ordering is the whole difference between the 39% and the 61%.

Element	What it is	When
Baseline	The number as it stands today — handling time, cost per case, error rate	Captured THIS month, before anything changes
One primary metric	The single number that moves if this works — e.g. cost per resolved ticket , owned by the head of support, not by the AI team	Named in advance. One, not five
Horizon	The earliest date that metric COULD move	Cost-to-serve: a quarter. Retention: not a quarter
Confound	The other thing that could explain a good result — written down before you start	A quiet quarter • a price change • your best team on the pilot
Counterfactual	A holdout or a staged rollout	Pre-read 1's experimentation platform, doing its job
Kill condition	The result that would make you stop	Agreed before the pilot starts

MIT's six-month P&L bar is this error institutionalised — holding a retention use case to a quarterly review kills it before it can succeed.

Takeaway If you cannot name the baseline, you have found the reason 39% is not higher.

After the break — you build it

Part B is the calculator. Three layers, every price read from configuration, and one number out.

What you will build. A cost-to-serve calculator that takes an architecture and a success rate and returns cost per successful task, the break-even, and a sensitivity table over the success rate.

How it runs: self-run — I walk a pre-run copy, you run your own afterwards. Vendor-neutral, your OpenRouter key.

Then you use it on something new. A second scenario you have not seen, with different volumes and a much cheaper failure. Same method, and — this is the point — the opposite answer.

Bring back: cost per successful task and the break-even. Not “ROI”, which is not a number until you define it.

Takeaway Fifteen minutes. Then the arithmetic that decides which architecture ships.

A 6-turn agent sends 45,000 input tokens: a 5,500-token prompt re-sent every turn, plus 800 tokens of growth per turn. Take it to 12 turns. Input tokens go to:

A · about 90,000 — it doubles

B · about 119,000 — about 2.6×

C · about 180,000 — about 4×

D · just under 90,000 — the fixed prompt dominates

Growth = the model's own reply plus the observation, re-sent as input on every later turn.

The calculator — three layers, kept apart

Each layer behaves differently as you scale, so mixing them is how a cost model stops being useful.

Layer	What goes in it	How it behaves
1 • Per-task variable	Fresh input + cached input ($\approx 0.1 \times$ input) + output + retrieval + tool-call fees. For an agent, SUM OVER TURNS — input accumulates	Linear in volume. Never amortises
2 • Per-task expected fallback	$(1 - \text{success rate}) \times \text{cost of handling a failure}$ — human minutes at a loaded rate, or a frontier retry	Usually the largest. Usually omitted
3 • Fixed monthly	Vector store, infrastructure, evaluation runs, monitoring, the fraction of an engineer who maintains it	Amortises. Gets cheaper per task forever

If you are pricing a product rather than costing a feature, Bessemer would add customer-success overhead and sales time to layer 3.

Takeaway Cost per successful task = layer 1 + layer 2, plus layer 3 spread over volume.

How this notebook runs

Same shape as Week 4: I walk a pre-run copy, you run your own. Nobody is blocked by a network or a key.

Self-run, vendor-neutral. BACKEND / MODEL / BASE_URL at the top; one function knows a vendor exists. Your own OpenRouter key, and the default path costs nothing because the prices are data, not API calls.

On screen now: the pre-run copy, so the arithmetic is visible even if the room's network is not.

Every price lives in one CONFIG block. Change a number there and the whole model re-costs. Nothing is hardcoded anywhere else in the notebook, and the block carries a dated comment saying where each price came from.

Why it matters: Gemini 3.7 Flash launched 13 Aug 2026 on a 50% introductory discount that expires 31 Dec 2026. A hardcoded price is already wrong.

Takeaway Prices are data. Put them in one place and date them.

Getting the arithmetic right

FOR BUILDERS

PRE-READ 3

Three places this calculation goes wrong in practice — all three are in the notebook as asserts.

One currency, and an agent input that is a SUM. Mixing a local-currency wage into US-dollar token prices is the most common error. And an agent's input is $6 \times \text{base} + \text{increment} \times (0+1+\dots+5)$, not $6 \times \text{base}$ — plus the model's own replies, re-billed each turn.

The tell: if your agent cost scales linearly with turns, you have modelled it wrong.

Report the break-even and the sensitivity, not a point estimate. The success rate is the term you are least sure of and the one the answer is most sensitive to. A single number invites false confidence; a table over ± 10 points invites a decision.

For builders: assert that the success rate is between 0 and 1, that prices are positive, and that no cost is negative. Cheap, and it catches typos.

Takeaway A cost model without a sensitivity table is a guess with decimal places.

PAIR EXERCISE • 6 MIN

Price this one yourself. Supplier-invoice matching, 12,000 invoices a month.

A • cheap model, one call + retrieval — \$0.0005 per task, resolves 75%

B • frontier model, 4-turn agent — \$0.123 per task, resolves 92%

A failure costs 45 seconds of a clerk at \$40/hour
= \$0.50

Give me: cost per successful task for each, and the break-even

6 minutes in pairs • the numbers are on the slide • we take two answers

Same method, opposite answer

In the worked example the expensive architecture won. Here the cheap one does. Nothing about the method changed.

What flipped it. The cost of being wrong. A support escalation costs \$6.00; a clerk glancing at a flagged invoice costs \$0.50 — twelve times less. So the 17-point resolution gain buys twelve times less.

The rule: the fallback cost decides. Not the model, not the architecture, not last year's answer.

Which is why you cannot carry a rule of thumb between problems. “Frontier models pay for themselves” is true where failure is expensive and false where it is cheap. Both sentences are wrong as general advice, and both are right somewhere.

What to carry instead: the three layers, the break-even, and the sensitivity. Those transfer; the answers do not.

$A = \$0.13 \cdot B = \$0.16 \cdot \text{break-even} \approx 67.5\%$. On this queue the difference is about \$5,400 a year — sometimes the calculator's job is to end the argument, not win it.

Takeaway Carry the method. Recompute the answer for every problem.

CAPSULE 2 • TAKEAWAY

Cost per successful task. Always.

Three layers, a break-even and a sensitivity table. The token price is the visible number; the failure cost is usually the real one — and it decides which architecture ships.